

ZEEWAY

Teste Técnico

Pessoa Desenvolvedora Full-Stack Pleno

PRAZO
1 dia

ETAPA 1
Desafio de código

ETAPA 2
Arquitetura sem código

Duas etapas independentes: implementação e raciocínio sistêmico

Contexto

Na Zeeway, buscamos pessoas desenvolvedoras capazes de transformar uma necessidade de produto em uma solução funcional, tomando decisões técnicas com autonomia e mantendo atenção à experiência do usuário.

Neste desafio, você deverá construir uma pequena aplicação web para gerenciamento e priorização de demandas de produto.

O objetivo não é desenvolver um sistema completo ou extremamente sofisticado. Queremos entender sua capacidade de compreender um problema, definir uma solução adequada e entregar uma aplicação funcional de ponta a ponta.

O teste está dividido em duas etapas independentes: uma entrega prática de código e uma etapa teórica de arquitetura, entregue em arquivo separado.

Prazo

O prazo para realização do desafio é de 1 dia.

Orientação de escopo

Controle o escopo e priorize o que gera mais valor para o produto. Caso não consiga implementar alguma funcionalidade, documente como ela seria desenvolvida.

ETAPA 1 - DESAFIO PRÁTICO DE CÓDIGO

Painel de Demandas de Produto

Uma equipe de produto precisa registrar, organizar e acompanhar solicitações de novas funcionalidades.

Cada demanda deverá conter:

- Título;
- Descrição;
- Solicitante;
- Nível de impacto;
- Nível de urgência;
- Status;
- Data de criação.

Os níveis de impacto e urgência devem ser classificados de 1 a 5.

$$\text{Prioridade} = (\text{Impacto} \times 2) + \text{Urgência}$$

Demandas com maior pontuação deverão aparecer primeiro por padrão.

Funcionalidades obrigatórias

1. Listagem de demandas

A tela principal deverá apresentar as demandas cadastradas.

Cada item deve exibir, no mínimo:

- Título;
- Solicitante;
- Impacto;
- Urgência;
- Pontuação de prioridade;
- Status;
- Data de criação.

A listagem deve ser ordenada pela prioridade, da maior para a menor.

2. Cadastro de demanda

O usuário deverá conseguir cadastrar uma nova demanda por meio de um formulário.

O formulário deverá possuir validações básicas, tanto no front-end quanto no back-end.

3. Atualização de status

O usuário deverá conseguir alterar o status de uma demanda.

Utilize os seguintes status:

- Pendente;
- Em andamento;
- Concluída;
- Cancelada.

4. Filtros

A aplicação deverá permitir filtrar as demandas por:

- Status;
- Solicitante;
- Impacto.

5. Resumo do produto

A tela principal deverá apresentar um pequeno resumo contendo:

- Total de demandas;
- Quantidade de demandas pendentes;
- Quantidade de demandas em andamento;
- Quantidade de demandas concluídas.

6. Persistência

As informações devem ser armazenadas em um banco de dados.

Pode ser utilizado SQLite, PostgreSQL ou outra opção devidamente justificada.

Requisitos técnicos

Front-end

O front-end deve ser desenvolvido utilizando:

- Next.js;
- React.js;
- TypeScript.

Esperamos encontrar:

- Componentes organizados;
- Comunicação adequada com a API;

- Estados de carregamento;
- Tratamento de erros;
- Estado vazio para listas sem resultados;
- Interface minimamente responsiva;
- Código legível e de fácil manutenção.

A escolha de biblioteca de componentes ou estilização é livre.

Back-end

O back-end deve ser desenvolvido em Python.

A escolha do framework é livre. Algumas opções possíveis:

- FastAPI;
- Django;
- Flask.

A API deverá disponibilizar operações para:

- Criar uma demanda;
- Listar demandas;
- Consultar uma demanda;
- Atualizar uma demanda;
- Alterar o status;
- Remover uma demanda.

Esperamos encontrar:

- Organização clara do projeto;
- Validação dos dados recebidos;
- Tratamento adequado de erros;
- Separação coerente de responsabilidades;
- Regras de negócio concentradas no back-end;
- Cálculo da prioridade realizado no servidor.

Arquitetura

A solução deve seguir uma abordagem de monólito, priorizando simplicidade, coesão e facilidade de evolução.

Não é necessário criar microsserviços, filas ou uma infraestrutura complexa.

A estrutura escolhida deve ser explicada brevemente no README.

Git

O projeto deverá ser disponibilizado em um repositório Git.

Esperamos:

- Commits com mensagens claras;
- Histórico que demonstre a evolução da solução;
- Ausência de credenciais ou informações sensíveis;
- Arquivo .gitignore configurado corretamente.

Evite entregar todo o desenvolvimento em apenas um commit.

Testes

Implemente ao menos:

- Um teste relacionado à regra de cálculo da prioridade;
- Um teste de endpoint ou serviço do back-end.

Testes adicionais serão considerados positivamente, mas não devem comprometer a entrega das funcionalidades principais.

Uso de Inteligência Artificial

O uso de ferramentas de inteligência artificial está liberado e é incentivado.

Você pode utilizar, por exemplo:

- ChatGPT;
- Claude;
- Cursor;
- GitHub Copilot.

Inclua no README uma pequena seção informando:

- Quais ferramentas foram utilizadas;
- Em quais partes do desenvolvimento elas ajudaram;
- Como você revisou ou validou o código gerado.

Não será avaliado negativamente o uso de IA. Queremos entender como você utiliza essas ferramentas para aumentar sua produtividade sem comprometer a qualidade da entrega.

Diferenciais

Os itens abaixo são opcionais:

- Implementação de paginação;
- Busca por título ou descrição;
- Ordenação configurável;
- Atualização otimista da interface;
- Docker ou Docker Compose;
- Documentação automática da API;
- Testes de front-end;
- Histórico de alterações de status;
- Deploy da aplicação;
- Uso de Elixir em alguma parte coerente da solução.

O uso de Elixir não é obrigatório. Caso seja utilizado, explique a decisão e evite aumentar desnecessariamente a complexidade do projeto.

Também será considerado como diferencial apresentar, no README, como você desenvolveria uma versão equivalente do back-end utilizando Elixir.

Entrega da Etapa 1

Envie:

- Link do repositório;
- Instruções para execução local;
- Arquivo README;
- Link da aplicação publicada, caso tenha realizado deploy.

O README deve conter:

1. Descrição da solução;
2. Tecnologias utilizadas;
3. Instruções de instalação e execução;
4. Decisões técnicas relevantes;
5. Limitações conhecidas;
6. Melhorias que seriam realizadas com mais tempo;
7. Ferramentas de IA utilizadas;
8. Tempo aproximado dedicado ao desafio.

A aplicação deve conseguir ser executada seguindo apenas as instruções disponíveis no repositório.

Observações importantes

- Não esperamos uma aplicação perfeita.
- Não é necessário implementar uma autenticação completa.
- Não é necessário criar uma infraestrutura de produção complexa.
- Bibliotecas externas podem ser utilizadas livremente.
- Decisões simples e bem justificadas são melhores do que soluções excessivamente sofisticadas.
- Caso precise escolher entre quantidade de funcionalidades e qualidade da entrega, priorize uma solução funcional, clara e bem documentada.

ETAPA 2 - QUESTÕES DE ARQUITETURA E RACIOCÍNIO SISTÊMICO

Esta etapa é totalmente independente do desafio de código.

Não é necessário implementar, adaptar ou demonstrar essas situações na aplicação desenvolvida na Etapa 1. O objetivo é avaliar como você estrutura problemas de concorrência, consistência, disponibilidade e escala.

ENTREGA SEPARADA

Responda às perguntas em um arquivo próprio, preferencialmente chamado ARQUITETURA.md. Não inclua as respostas no README da aplicação e não é necessário escrever código para esta etapa.

Perguntas

1. Você tem um recurso finito - por exemplo, 100 conexões, 100 tokens ou 100 slots - e existe constantemente mais demanda do que oferta. Como você começa a pensar a solução?
2. Dois clientes pedem o último slot disponível ao mesmo tempo. Como você garante que apenas um deles recebe o recurso?
3. O recurso pode ser devolvido pelo cliente ou expirar sozinho depois de determinado período. Isso muda sua arquitetura? Explique.
4. Se o serviço que controla o pool cair no meio da operação, o que acontece com os recursos que já foram emprestados?
5. Se uma segunda instância da aplicação for adicionada atrás de um load balancer, como garantir que o limite global de 100 recursos continue sendo respeitado?
6. Não há recurso disponível agora, mas um está prestes a ser devolvido. O cliente recebe erro imediato ou espera? Quais fatores influenciam essa decisão?

Pontos que podem ser abordados

- Concorrência e condições de corrida;
- Consistência e atomicidade;
- Reserva, devolução e expiração de recursos;
- Tolerância a falhas e recuperação;
- Escalabilidade horizontal e estado compartilhado;
- Filas, espera, timeout e políticas de rejeição;
- Decisões adotadas, alternativas consideradas e trade-offs.

Boa sorte!